

1. Write a program to implement following vector operations: at(i), insert (i,e), erase(i,e), set(i,e), push_back(), pop_back()

```
#include <iostream>

#include <vector>
using namespace std;

/* 1. Write a program to implement following vector operations: at(i), insert (i,e),
erase(i,e), set(i,e), push_back(), pop_back(). */

void printVector(const string& operation, const vector<int>& vec) {
    cout << operation << ": [ ";
    for (int val : vec) {
        cout << val << " ";
    }
    cout << "]\n\n";
}

int main() {
    vector<int> v;
    cout << "=== Vector Operations Demonstration ===\n\n";

    // 1. push_back(e) - Adds an element to the end
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);
    v.push_back(40);
    v.push_back(50);
    printVector("1. After push_back (added 10, 20, 30, 40, 50)", v);

    // 2. pop_back() - Removes the last element
    v.pop_back();
    printVector("2. After pop_back (removed last element)", v);

    // 3. at(i) - Accesses the element at index i (throws an error if out of bounds)
    cout << "3. Using at(i): Element at index 2 is " << v.at(2) << "\n\n";

    // 4. set(i, e) - C++ uses assignment to set a new value at an index
    // Setting element at index 1 to 99
    v.at(1) = 99; // Alternately, you can use: v[1] = 99;
    printVector("4. After 'set(1, 99)' via v.at(1) = 99", v);

    // 5. insert(i, e) - Inserts element 'e' at index 'i'
    // C++ requires an iterator (v.begin() + i) rather than just an integer
    int insertIndex = 2;
    int insertValue = 100;
    v.insert(v.begin() + insertIndex, insertValue);
    printVector("5. After 'insert(2, 100)'", v);

    // 6. erase(i) - Erases an element at a specific index
    int eraseIndex = 3;
    v.erase(v.begin() + eraseIndex);
    printVector("6a. After 'erase(3)' (removed element at index 3)", v);
```

```

// 6b. erase(start, end) - Erases a range of elements from start to end (exclusive)
// erasing from index 1 up to index 3 (erases index 1 and 2)
int start = 1;
int end = 3;
v.erase(v.begin() + start, v.begin() + end);
printVector("6b. After 'erase(1, 3)' (removed range from index 1 up to 3)", v);

return 0;
}

```

Output:

```

PS C:\Users\HP\Downloads\New folder (2)> cd "c:\Users\HP\Downloads\New folder (2)\\" ; if ($?) { g++ 1.cpp -o 1 } ; if ($?) { .\1 }
=== Vector Operations Demonstration ===

1. After push_back (added 10, 20, 30, 40, 50): [ 10 20 30 40 50 ]
2. After pop_back (removed last element): [ 10 20 30 40 ]
3. Using at(i): Element at index 2 is 30
4. After 'set(1, 99)' via v.at(1) = 99: [ 10 99 30 40 ]
5. After 'insert(2, 100)': [ 10 99 100 30 40 ]
6a. After 'erase(3)' (removed element at index 3): [ 10 99 100 40 ]
6b. After 'erase(1, 3)' (removed range from index 1 up to 3): [ 10 40 ]

```

2. Write a cpp program to implement a vector and perform extended array (Insertion, deletion) operations using vector class.

```

#include <iostream>
#include <limits>
using namespace std;

template<class T>
class Vector {
    int size;
    int capacity;
    T* arr;

    void doublingSystem() {
        cout << "-> Capacity Exceeded. Time to Double the Capacity!" << endl;
        capacity *= 2;

        T* temp = new T[capacity];
        for (int i = 0; i < size; i++) {
            *(temp + i) = *(arr + i);
        }
        delete[] arr;
        arr = temp;
    }

public:
    ~Vector() {

```

```

        delete[] arr;
    }

    Vector() {
        size = 0;
        capacity = 1;
        arr = new T[capacity];
    }

    Vector(int x) {
        if (x < 0) {
            size = 0;
            capacity = 1;
            arr = new T[capacity];
            return;
        }
        size = x;
        capacity = x;
        arr = new T[capacity];
    }

    Vector(int x, T* list) {
        if (list == nullptr) {
            throw invalid_argument("List is a nullptr");
        }
        if (x < 0) {
            size = 0;
            capacity = 1;
            arr = new T[capacity];
            return;
        }

        size = x;
        capacity = x;
        arr = new T[capacity];

        for (int i = 0; i < x; i++) *(arr + i) = *(list + i);
    }

    T at(int i) {
        if (i < 1 || i > size) {
            cout << "-> Error: Position out of bounds!" << endl;
            return -1;
        }
        return *(arr + i - 1);
    }

    void insert(int i, T e) {
        if (i < 1 || i > size + 1) {
            cout << "-> Error: Position out of bounds!" << endl;
            return;
        }
        if (size == capacity) doublingSystem();
    }

```

```

        for (int it = size; it > i - 1; it--) {
            *(arr + it) = *(arr + it - 1);
        }

        *(arr + i - 1) = e;
        size++;
    }

    void erase(int i) {
        if (i < 1 || i > size) {
            cout << "-> Error: Position out of bounds!" << endl;
            return;
        }
        if (size == 0) {
            cout << "-> Error: Vector is empty!" << endl;
            return;
        }

        for (int it = i - 1; it < size - 1; it++) {
            *(arr + it) = *(arr + it + 1);
        }
        size--;
    }

    void set(int i, T e) {
        if (i < 1 || i > size) {
            cout << "-> Error: Position out of bounds!" << endl;
            return;
        }
        *(arr + i - 1) = e;
    }

    void push_back(T e) {
        if (size == capacity) doublingSystem();
        *(arr + size) = e;
        size++;
    }

    void pop_back() {
        if (size == 0) {
            cout << "-> Error: Vector is already empty!" << endl;
            return;
        }
        size--;
    }

    void print() {
        cout << "Vector: [ ";
        for (int i = 0; i < size; i++) {
            cout << *(arr + i) << " ";
        }
        cout << "]" (Size: " << size << ", Capacity: " << capacity << ") \n";
    }
};

```

```

int main() {
    Vector<int> v;
    int choice, pos, value;

    while (true) {
        cout << "\n=== Custom Vector Menu (1-Based Positions) ===\n";
        cout << "1. Push Back (push_back)\n";
        cout << "2. Pop Back (pop_back)\n";
        cout << "3. Insert at Position (insert)\n";
        cout << "4. Erase at Position (erase)\n";
        cout << "5. Set Element at Position (set)\n";
        cout << "6. Get Element at Position (at)\n";
        cout << "7. Print Vector\n";
        cout << "8. Exit\n";
        cout << "Enter your choice (1-8): ";

        if (!(cin >> choice)) {
            cout << "Invalid input! Please enter a number.\n";
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }

        if (choice == 8) {
            cout << "Exiting program...\n";
            break;
        }

        switch (choice) {
            case 1:
                cout << "Enter value to push back: ";
                cin >> value;
                v.push_back(value);
                v.print();
                break;

            case 2:
                v.pop_back();
                v.print();
                break;

            case 3:
                cout << "Enter position (1 to " << (v.at(1) == -1 ? 1 : 999) <<
/*placeholder*/ ") to insert at: ";
                cin >> pos;
                cout << "Enter value to insert: ";
                cin >> value;
                v.insert(pos, value);
                v.print();
                break;

            case 4:
                cout << "Enter position to erase (1-based): ";

```

```

        cin >> pos;
        v.erase(pos);
        v.print();
        break;

    case 5:
        cout << "Enter position to set (1-based): ";
        cin >> pos;
        cout << "Enter new value: ";
        cin >> value;
        v.set(pos, value);
        v.print();
        break;

    case 6:
        cout << "Enter position to retrieve (1-based): ";
        cin >> pos;
        value = v.at(pos);
        if (value != -1) {
            cout << "-> Value at position " << pos << " is: " << value << "\n";
        }
        break;

    case 7:
        v.print();
        break;

    default:
        cout << "Invalid choice! Please select an option between 1 and 8.\n";
    }
}

return 0;
}

```

Output:

```
PS C:\Users\HP\Downloads\New folder (2)> cd "c:\Users\HP\Downloads\New folder (2)\"; if ($?) { g++ 2.cpp -o 2 } ; if ($?) { .\2 }
```

```
=== Custom Vector Menu (1-Based Positions) ===
```

```
1. Push Back (push_back)
2. Pop Back (pop_back)
3. Insert at Position (insert)
4. Erase at Position (erase)
5. Set Element at Position (set)
6. Get Element at Position (at)
7. Print Vector
8. Exit
Enter your choice (1-8): 1
Enter value to push back: 1
Vector: [ 1 ] (Size: 1, Capacity: 1)
```

```
=== Custom Vector Menu (1-Based Positions) ===
```

```
1. Push Back (push_back)
2. Pop Back (pop_back)
3. Insert at Position (insert)
4. Erase at Position (erase)
5. Set Element at Position (set)
6. Get Element at Position (at)
7. Print Vector
8. Exit
Enter your choice (1-8): 1
Enter value to push back: 2
-> Capacity Exceeded. Time to Double the Capacity!
Vector: [ 1 2 ] (Size: 2, Capacity: 2)
```

```
=== Custom Vector Menu (1-Based Positions) ===
```

```
1. Push Back (push_back)
2. Pop Back (pop_back)
3. Insert at Position (insert)
4. Erase at Position (erase)
5. Set Element at Position (set)
6. Get Element at Position (at)
7. Print Vector
8. Exit
Enter your choice (1-8): 2
Vector: [ 1 ] (Size: 1, Capacity: 2)
```

```
=== Custom Vector Menu (1-Based Positions) ===
```

```
1. Push Back (push_back)
2. Pop Back (pop_back)
3. Insert at Position (insert)
4. Erase at Position (erase)
5. Set Element at Position (set)
6. Get Element at Position (at)
7. Print Vector
8. Exit
Enter your choice (1-8): 3
Enter position (1 to 999) to insert at: 2
Enter value to insert: 4
Vector: [ 1 4 ] (Size: 2, Capacity: 2)
```

```
=== Custom Vector Menu (1-Based Positions) ===
```

```
1. Push Back (push_back)
2. Pop Back (pop_back)
3. Insert at Position (insert)
4. Erase at Position (erase)
5. Set Element at Position (set)
6. Get Element at Position (at)
7. Print Vector
8. Exit
Enter your choice (1-8): 4
Enter position to erase (1-based): 1
Vector: [ 4 ] (Size: 1, Capacity: 2)
```

```
=== Custom Vector Menu (1-Based Positions) ===
```

```
1. Push Back (push_back)
2. Pop Back (pop_back)
3. Insert at Position (insert)
4. Erase at Position (erase)
5. Set Element at Position (set)
6. Get Element at Position (at)
7. Print Vector
8. Exit
Enter your choice (1-8): 5
Enter position to set (1-based): 1
Enter new value: 7
Vector: [ 7 ] (Size: 1, Capacity: 2)
```

=== Custom Vector Menu (1-Based Positions) ===

1. Push Back (push_back)
2. Pop Back (pop_back)
3. Insert at Position (insert)
4. Erase at Position (erase)
5. Set Element at Position (set)
6. Get Element at Position (at)
7. Print Vector
8. Exit

Enter your choice (1-8): 6

Enter position to retrieve (1-based): 1

-> Value at position 1 is: 7

=== Custom Vector Menu (1-Based Positions) ===

1. Push Back (push_back)
2. Pop Back (pop_back)
3. Insert at Position (insert)
4. Erase at Position (erase)
5. Set Element at Position (set)
6. Get Element at Position (at)
7. Print Vector
8. Exit

Enter your choice (1-8): 7

Vector: [7](Size: 1, Capacity: 2)

=== Custom Vector Menu (1-Based Positions) ===

1. Push Back (push_back)
2. Pop Back (pop_back)
3. Insert at Position (insert)
4. Erase at Position (erase)
5. Set Element at Position (set)
6. Get Element at Position (at)
7. Print Vector
8. Exit

Enter your choice (1-8): 8

Exiting program...

3. Given a vector of integers and a target value, return the indices of two numbers whose sum equals the target eg: nums = [2, 7, 11, 15]. target = 9.

```
#include <iostream>
#include <vector>
#include <unordered_map>

std::vector<int> twoSum(const std::vector<int>& nums, int target) {
    std::unordered_map<int, int> num_map;

    for (int i = 0; i < nums.size(); ++i) {
        int complement = target - nums[i];

        if (num_map.find(complement) != num_map.end()) {
            return {num_map[complement], i};
        }

        num_map[nums[i]] = i;
    }

    return {};
}

int main() {
    std::vector<int> nums = {2, 7, 11, 15};
    int target = 9;

    std::vector<int> result = twoSum(nums, target);

    if (!result.empty()) {
        std::cout << result[0] << " " << result[1] << std::endl;
    } else {
        std::cout << "No pair found" << std::endl;
    }

    return 0;
}
```

Output:

```
PS C:\Users\HP\Downloads\New folder (2)> cd "c:\Users\HP\Downloads\New folder (2)\"; if ($?) { g++ 3.cpp -o 3 }; if ($?) { .\3 }
0 1
```

4. Write a menu-driven C++ program that performs the following operations on a vector:

1. Display the vector
2. Rotate left by one position
3. Rotate right by one position
4. Rotate left by K positions
5. Rotate right by K positions
6. Exit

```
#include <iostream>
#include <vector>
#include <algorithm>

void displayVector(const std::vector<int>& vec) {
    if (vec.empty()) {
        std::cout << "Vector is empty.\n";
        return;
    }
    std::cout << "Vector elements: ";
    for (int val : vec) {
        std::cout << val << " ";
    }
    std::cout << "\n";
}

void rotateLeft(std::vector<int>& vec, int k) {
    if (vec.empty()) return;
    int n = vec.size();
    k = k % n; // Handle cases where k >= vector size
    std::rotate(vec.begin(), vec.begin() + k, vec.end());
}

void rotateRight(std::vector<int>& vec, int k) {
    if (vec.empty()) return;
    int n = vec.size();
    k = k % n; // Handle cases where k >= vector size
    std::rotate(vec.rbegin(), vec.rbegin() + k, vec.rend());
}

int main() {
    int n;
    std::cout << "Enter initial size of vector: ";
    std::cin >> n;

    std::vector<int> vec(n);
    std::cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; ++i) {
        std::cin >> vec[i];
    }
}
```

```

int choice;
do {
    std::cout << "\n=====\\n";
    std::cout << "1. Display the vector\\n";
    std::cout << "2. Rotate left by 1 position\\n";
    std::cout << "3. Rotate right by 1 position\\n";
    std::cout << "4. Rotate left by K positions\\n";
    std::cout << "5. Rotate right by K positions\\n";
    std::cout << "6. Exit\\n";
    std::cout << "Enter your choice: ";
    std::cin >> choice;

    switch (choice) {
        case 1:
            displayVector(vec);
            break;
        case 2:
            rotateLeft(vec, 1);
            std::cout << "Vector rotated left by 1 position.\\n";
            break;
        case 3:
            rotateRight(vec, 1);
            std::cout << "Vector rotated right by 1 position.\\n";
            break;
        case 4: {
            int k;
            std::cout << "Enter value for K: ";
            std::cin >> k;
            rotateLeft(vec, k);
            std::cout << "Vector rotated left by " << k << " position(s).\\n";
            break;
        }
        case 5: {
            int k;
            std::cout << "Enter value for K: ";
            std::cin >> k;
            rotateRight(vec, k);
            std::cout << "Vector rotated right by " << k << " position(s).\\n";
            break;
        }
        case 6:
            std::cout << "Exiting program.\\n";
            break;
        default:
            std::cout << "Invalid choice! Please select a valid option.\\n";
    }
} while (choice != 6);

return 0;
}

```

Output:

```
PS C:\Users\HP\Downloads\New folder (2)> cd "c:\Users\HP\Downloads\New folder (2)\\" ; if ($?) { g++ 4.cpp -o 4 } ; if ($?) { .\4 }
Enter initial size of vector: 4
Enter 4 elements: 1 2 3 4

=====
1. Display the vector
2. Rotate left by 1 position
3. Rotate right by 1 position
4. Rotate left by K positions
5. Rotate right by K positions
6. Exit
Enter your choice: 1
Vector elements: 1 2 3 4

=====
1. Display the vector
2. Rotate left by 1 position
3. Rotate right by 1 position
4. Rotate left by K positions
5. Rotate right by K positions
6. Exit
Enter your choice: 2
Vector rotated left by 1 position.

=====
1. Display the vector
2. Rotate left by 1 position
3. Rotate right by 1 position
4. Rotate left by K positions
5. Rotate right by K positions
6. Exit
Enter your choice: 1
Vector elements: 2 3 4 1

=====
1. Display the vector
2. Rotate left by 1 position
3. Rotate right by 1 position
4. Rotate left by K positions
5. Rotate right by K positions
6. Exit
Enter your choice: 3
Vector rotated right by 1 position.

=====
1. Display the vector
2. Rotate left by 1 position
3. Rotate right by 1 position
4. Rotate left by K positions
5. Rotate right by K positions
6. Exit
Enter your choice: 1
Vector elements: 1 2 3 4
```

```
=====
1. Display the vector
2. Rotate left by 1 position
3. Rotate right by 1 position
4. Rotate left by K positions
5. Rotate right by K positions
6. Exit
```

Enter your choice: 4

Enter value for K: 5

Vector rotated left by 5 position(s).

```
=====
1. Display the vector
2. Rotate left by 1 position
3. Rotate right by 1 position
4. Rotate left by K positions
5. Rotate right by K positions
6. Exit
```

Enter your choice: 1

Vector elements: 2 3 4 1

```
=====
1. Display the vector
2. Rotate left by 1 position
3. Rotate right by 1 position
4. Rotate left by K positions
5. Rotate right by K positions
6. Exit
```

Enter your choice: 5

Enter value for K: 6

Vector rotated right by 6 position(s).

```
=====
1. Display the vector
2. Rotate left by 1 position
3. Rotate right by 1 position
4. Rotate left by K positions
5. Rotate right by K positions
6. Exit
```

Enter your choice: 1

Vector elements: 4 1 2 3

```
=====
1. Display the vector
2. Rotate left by 1 position
3. Rotate right by 1 position
4. Rotate left by K positions
5. Rotate right by K positions
6. Exit
```

Enter your choice: 6

Exiting program.

5. Write a C++ program to remove duplicate elements from a sorted vector of integers.

```
#include <iostream>
#include <vector>
#include <algorithm>

void removeDuplicates(std::vector<int>& nums) {
    auto it = std::unique(nums.begin(), nums.end());
    nums.erase(it, nums.end());
}

int main() {
    std::vector<int> nums = {1, 1, 2, 2, 3, 4, 4, 4, 5, 5};

    std::cout << "Original vector: ";
    for (int num : nums) {
        std::cout << num << " ";
    }
    std::cout << "\n";

    removeDuplicates(nums);

    std::cout << "Vector after removing duplicates: ";
    for (int num : nums) {
        std::cout << num << " ";
    }
    std::cout << "\n";

    return 0;
}
```

Output:

```
PS C:\Users\HP\Downloads\New folder (2)> cd "c:\Users\HP\Downloads\New folder (2)\"; if ($?) { g++ 5.cpp -o 5 }; if ($?) { .\5 }
Original vector: 1 1 2 2 3 4 4 4 5 5
Vector after removing duplicates: 1 2 3 4 5
```